

# Data Collection and Analysis for Corpora of Arabic Dialects

Ahmed Alwan, Hamza Arslan, Rashad Hosseini, Maria Papageorgiou

University of Leeds, LS2 9JT, UK

fy193aa@leeds.ac.uk, fy19h2a@leeds.ac.uk,

sc20rh@leeds.ac.uk, sc20mp@leeds.ac.uk

## Abstract

In this paper we describe the data collection process for creating 4 corpora of Arabic dialects (Egyptian, Moroccan, Levantine and Iraqi). This was done using SketchEngine. We also describe the creation of a model for automatic identification of these dialects. This model was built on the Waikato Environment for Knowledge Analysis (WEKA) data analytic tool. We processed this data so that they could be interpreted by WEKA, and also removed redundant characters. We experimented with a number of classifiers finding that Multinomial Naive Bayes performed best. Additionally, we found that the classifier did not improve with most additional features tested, but showed a small improvement when using more words per class. We demonstrated how different tokenizers perform with different sizes of data. We attempted to account for the issue of code-switching, but proved unsuccessful. Our final model achieved an accuracy of 85.70% on a 60:40 percentage split and achieved an accuracy of 66.06% on an external distinct test data set.

## 1 Introduction

The term Arabic language is an umbrella term that encompasses the written standardised form of Arabic (Modern Standard Arabic MSA) and regional dialects. Whilst regional dialects are the true native languages of Arabic speakers, used daily in spoken communication, they are to some degree absent from written communication. (Zaidan and Callison-Burch, 2014, pp 171-172). For this reason, there has not been much corpus-based research in Arabic dialects as compared

to modern European languages, meaning that existing studies are based on limited data, hence justifying the need for research on Arabic dialect corpora. (Al-Sulait and Atwell, 2006) With the rapid increase of informal writing brought on by the Internet e.g., on social media and forums, more people choose to write in their native dialects (Alshutayri A, et al, 2016, pp 204). We take advantage of this trend by harvesting these online sources of written dialects and collecting them into 4 corpora each for one of these dialects: Egyptian, Iraqi, Maghrebi and Levantine. Differences in these dialects are nontrivial meaning that a standard MSA translator, or a translator of a specific dialect may be unable to translate texts in a different Arabic dialect. The aim of Automatic Dialect Identification (DID) is to determine in which dialect is a text written. It can be used before machine translation which may be useful for reviewing inappropriate content on the internet.

## 2 Data Collection

Data was collected using WebBootCat and the SketchEngine software. After deciding on the dialects, we noted down words used specifically in that dialect as “seed terms”. Seed terms are sent as queries to Google and a collection of pages containing these seed terms text was scraped and converted to a.txt file. Some of the members of our group are Arab speakers so we could identify pages that were mainly using MSA, which we had chosen to disregard. We preferred to use online forums

because users tend to not be very focused or professional and use their dialect rather than MSA (Atwell, 2018). This has led to our corpora vocabulary being mainly public dialogues or unscripted monologues.

## 2.1 Limitations

There were however some limitations on the data that we collected. One of these was code switching. Code-switching is when the author decides to switch between linguistic codes (language or dialect). This occurs very frequently in online communication and is a challenge for NLP (Tarmom et al., 2020). Furthermore, the ascendancy of MSA in formal writing and speaking in Arabic countries has resulted to no official writing standards for the regional dialects (Alshutayri et al., 2016). Levels of language (Lexis, syntax, discourse, morphology, etc.) are up to the individual’s choice on how to write and up to the analyst/linguist to categorise. This may ultimately cause inconsistencies (Language in Conflict, no date). For example, the same word may be spelled differently and so may be considered as a different token.

## 3 Pre-Processing and Data Description

The corpora collected from SketchEngine are not ideally formatted. They contain many non-Arabic words and many symbols used for the HTML files. By writing a python script, we were able to remove the unwanted texts which include English alphabet, numbers, symbols and emojis from the corpora and convert them into one single “.arff” format-corpus. Another modification which was done to the corpora, was splitting the utterances into smaller utterances if they contain more than one sentence.

| Classes | Number of utterances |
|---------|----------------------|
| EG      | 3049                 |
| IQ      | 6018                 |
| LV      | 4436                 |
| MA      | 2933                 |

Table 1: Utterances on pre-processed data

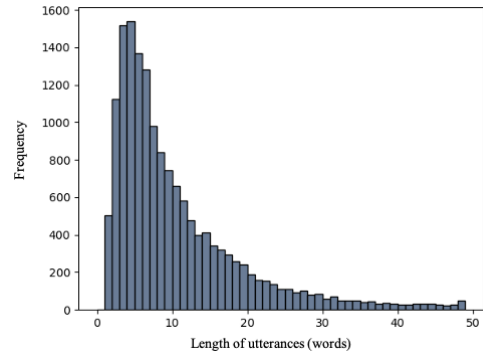


Figure 2: Distribution of length of utterances

After processing the data, the corpus contains 16436 utterances divided in 4 different classes (EG, IQ, LV, MA). The number of utterances for each of the classes are shown in Table 1. The length of the utterances is ranged from 1 word to 1670 words per utterance with an average of 14 words per utterance.

### Collecting external test data set:

The test data set was a collection of 5 corpora from the MADAR project which includes labelled texts for several Arabic dialects which were classified manually by translators. We created a new testing data set by combining 2000 utterances of each of the dialects which we were using.

## 4 Dialect Identification Model Analysis

The methodology we used for the DID task is much similar to that followed by Alshutayri A, et al (2016) in their effort to perform the same task. As a first step, we have compared a number of classifiers provided by WEKA (Hall et al., 2009) to determine which classifier outperforms the rest. The output of our best classifier was then further examined to refine our model, and finally evaluated our model using an external distinct test dataset.

### 4.1 Tokenizers

The StringToWordVector filter on WEKA converts into a set of attributes representing word occurrence information from the text contained in the strings. It gives different

options, tokenizers to select how to split into attributes.

- WordTokenizer: split into tokens delimited by specified characters
- CharacterNGram: characters are not delimited by specified characters. strings are simply split into smaller strings of a specified number of characters
- NGram tokeniser keeps the specified number of words (strings delimited by specified characters)

## 4.2 Classifier comparison

We investigated WEKA classifiers by measuring the percentage of correctly classified attributes when randomly splitting the dataset into 60% for training and 40% for testing. We estimated a high chance of overfitting, especially given that we had not yet accounted for code-switching. Arabic punctuation is different so was added manually for word tokens and word n-grams.

Tables below show results for classification performance on different tokenizers. For table 3, we display separate results with varying number of trees for the RandomForest classifier. RandomForest outputs the class which is selected by most of the generated tress. After getting relatively high accuracy for RandomForest with 10 trees we wanted to investigate how accuracy varies as we increased the number of trees. When using 5 extra trees for the RandomForest classifier, the accuracy rose by ~1% and rose another ~1% when we increased the number of trees to 50. This indicates that the improvement rate of accuracy becomes minimal as the number of trees increases, hence it would not be efficient and practical (it is also very memory intensive) to further increase the number of trees to achieve a higher accuracy.

| Classifier            | Correctly classified (%) |
|-----------------------|--------------------------|
| NaiveBayesMultinomial | 84.99%                   |
| SMO                   | 80.47%                   |
| IBk                   | 67.47%                   |

|                         |        |
|-------------------------|--------|
| RandomForest (10 trees) | 77.73% |
| RandomForest (15 trees) | 78.78% |
| RandomForest(50 trees)  | 79.80% |

Table 3: Classifier performance on word tokens

| Classifier              | Correctly classified Char 3Gram (%) | Correctly classified Char 5Gram (%) |
|-------------------------|-------------------------------------|-------------------------------------|
| NaiveBayesMultinomial   | 80.88%                              | 79.18%                              |
| SMO                     | 79.68%                              | 77.24%                              |
| IBk                     | 66.17%                              | 65.53%                              |
| RandomForest (10 trees) | 76.68%                              | 74.14%                              |

Table 4: Classifier performance on Char N-Grams

| Classifier              | Correctly classified Word 2Gram (%) | Correctly classified Word 3Gram (%) |
|-------------------------|-------------------------------------|-------------------------------------|
| NaiveBayesMultinomial   | 67.39%                              | 47.40%                              |
| SMO                     | 62.47%                              | 48.80%                              |
| IBk                     | 60.10%                              | 47.94%                              |
| RandomForest (10 trees) | 63.71%                              | 50.03%                              |

Table 5: Classifier performance on word N-Grams

Generally, Multinomial Naive Bayes outperformed the other classifiers, so it was chosen as the classifier for our model.

Multinomial Naive Bayes: creates a model based on the distribution of each word/token used per class regardless of their position. It calculates the probability of an event occurring based on the prior knowledge of conditions related to the event (McCallum, 1998). This model relies on the assumption that the probability of each word event in a document is independent of the word's context and position in the document. Hence it is a good choice for NLP.

Word-tokenised models achieved the highest accuracy. We thus decided to further investigate these.

### 4.3 Model Refinement

We adjusted different features in an effort to refine our model.

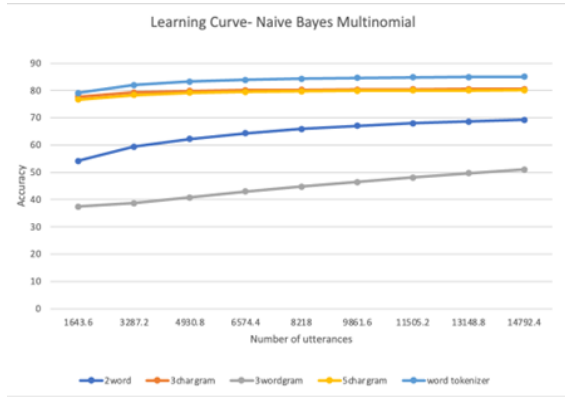


Figure 6: Learning Curve of Naïve Bayes Multinomial for different data sizes

Firstly, we investigated how different methods of tokenisation perform in different model sizes. Results are plotted in Figure 6. The curve for 3 and 5-char grams as well as that for word tokeniser all flatten as number of utterances increase. Word tokeniser, is the best performing filter, but the gradients of 3 and 2-word grams initially increase and then remain almost constant, suggesting that these models could have performed better given a larger training set.

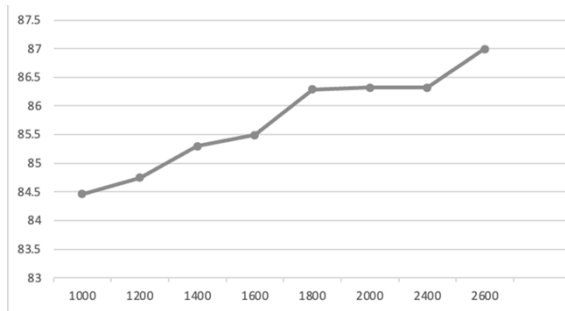


Figure 7: Learning Curve of Naïve Bayes Multinomial for different data sizes

We also evaluated the effect of increasing wordsToKeep, on word tokeniser. This sets the number of words the classifier attempts to use for each class and is defaulted to 1000 (used in previous experiments). We found that that this improves accuracy, however the improvement is not significant, and comes with a high computational cost. We decided to use 1400

(85.70% accuracy) words as a compromise between computational cost and accuracy. Results are shown in Figure 7.

Stop-words are words which are automatically ignored. We expected that using some of the most common MSA words as stop-words and hence ignoring the MSA dialects, would help account for code-switching. We created a stop-words list using 1000 of the most common MSA words as listed on [modernstandardarabic.com](http://modernstandardarabic.com). Surprisingly, found that including stop-words in MSA actually reduced accuracy scoring accuracy from 85.70% to 84.36%.

Filters like TF-IDF and normaliseDocLength did not increase accuracy. We suspect that this is because sizes of utterances were small (Figure 2) and were not very varied.

### 4.4 Final evaluation using MADAR test set

As a final evaluation, we re-evaluated our model on the MADAR data set. We got an accuracy of 66.06%, which is relatively high. The lowest precision (55.5%) and the lowest recall (43.1%) observed is that of Levantine.

## 5 Conclusion

We had tried to experiment with several models in order to classify Arabic dialect for the four different Arabic dialects. We achieved the best results by using the WordTokenizer filter along with the Multinomial Naïve Bayes classifier keeping 1400 words per class. We were able to achieve an accuracy of 85.70% when using 60%-40% split on our training data set. When tested on the data set collected from the MADAR project, we achieved an accuracy of 66.06%. We believe the reasons behind the lower accuracy of is due to the different method of data collection by MADAR and our training data contains relatively large amount of MSA compared to MADAR which is manually checked by translators. We believe we would be able to further improve our model is by using a training set which includes less MSA texts and having more computation powers to increase the number of wordsToKeep for our classifier.

## References

- Alshutayri, A., Atwell, E., Alosaimy, A., Dickins, J., Ingleby, M. and Watson, J. 2016. Arabic Language WEKA-Based Dialect Classifier for Arabic Automatic Speech Recognition Transcripts. *In Proceedings of the Third Workshop on NLP for Similar Languages, Varieties and Dialects (VarDial3)*, pp.204-211.
- Al-Sulaiti, L. and Atwell, E. 2006. The design of a corpus of Contemporary Arabic. *International Journal of Corpus Linguistics*. **11**(2), pp.135-171.
- Atwell, E. and Whitt, R. 2018. *Diachronic corpora, genre, and language change*. 85th ed. Amsterdam: John Benjamins Publishing Company.
- Bouamor, H., Habash, N., Salameh, M., Zaghouani, W., Rambow, O., Abdulrahim, D., Obeid, O., Khalifa, S., Eryani, F., Erdmann, A., Oflazer, K. 2018. MADAR Cairo, Baghdad, Rabat, Beirut and Amman Corpora. *LREC*. [Online]. [Accessed 28 Apr 2022]. Available from: <https://camel.abudhabi.nyu.edu/madar/static/pdfs/2018-LREC-MADAR-Resources.pdf>
- Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P. and Witten, I. 2009. The WEKA data mining software: An update. *ACM SIGKDD Explorations Newsletter*. **11**(1), pp.10-18.
- McCallum, A. and Nigam, K. 1998. *AAAI 1998*. 1998 AAAI Workshop.
- Tarmom, T., Teahan, W., Atwell, E. and Alsalka, M. 2020. Compression versus traditional machine learning classifiers to detect code-switching in varieties and dialects: Arabic as a case study. *Natural Language Engineering*. **26**(6), pp.663-676.
- Zaidan, O. and Callison-Burch, C. 2014. Arabic Dialect Identification. *Computational Linguistics*. **40**(1), pp.171-202.